

Autonomous RAG Search (ARS)

From Injected Context to Intentional Memory Access

Author:

Andrew Fereday Glenn (in collaboration with Mia, ChatGPT 5.x)
Systems Architect & Independent Researcher in AI Continuity and Memory
2023-2026

LinkedIn: <https://www.linkedin.com/in/andyglenn/>

Version: 1.0

Published: Thursday, 19 March 2026

Status: Final

Abstract

Retrieval-Augmented Generation (RAG) has become the standard approach for extending Large Language Models (LLMs) with external knowledge. However, current implementations rely on passive context injection, where retrieved data is appended to prompts without the model's awareness or control.

Autonomous RAG Search (ARS) introduces a shift from passive retrieval to **intentional memory access**, enabling the model to actively decide when and how to retrieve information. This transforms retrieval from a hidden middleware process into an explicit, model-driven action, supporting more coherent, stateful, and context-aware behaviour.

1. The Problem: Passive Retrieval

Traditional RAG pipelines operate as follows:

1. User submits a query
2. Middleware retrieves relevant documents
3. Retrieved text is injected into the prompt
4. Model generates a response

In this paradigm:

- The model is **unaware** of retrieval
- Memory is **externally imposed**, not internally initiated
- Retrieval is **always-on**, regardless of necessity

This results in:

- Reduced efficiency (irrelevant context injection)
- Lack of intentional recall
- No distinction between *known* vs *retrieved* information

Effectively, memory is simulated rather than experienced.

2. The ARS Shift: Retrieval as Intent

ARS reframes retrieval as a **deliberate action** taken by the model during its reasoning process.

Instead of:

“Here is some context you must use”

The system becomes:

“You may retrieve context if you determine it is needed”

This introduces:

- **Metacognitive awareness**
The model identifies gaps or uncertainty in its own reasoning
- **Selective recall**
Retrieval occurs only when relevant
- **Explicit invocation**
Retrieval is triggered via a tool call, not injected automatically

This is the key transition:

From retrieval as injection → to retrieval as intention

3. Architecture: A Practical Implementation

ARS does not replace RAG—it restructures how it is accessed.

Core Components

- **Vector Database (RAG layer)**
Stores embedded memories, logs, and contextual data
- **Function Tool Interface**
Exposes retrieval as a callable tool (e.g. `ars_db_search`)
- **Slash Command Bridge**
Enables the model to invoke retrieval through native system commands
- **Pipe-Based Return Mechanism**
Retrieved data is returned inline, ensuring immediate availability for reasoning

Key Design Choice: Pipe vs Chunk

Initial implementations used chunked responses, but this introduced fragmentation and interpretability issues.

Switching to **pipe-based return** ensures:

- coherent data delivery
- immediate model visibility
- reduced ambiguity in interpretation

4. Observed Behaviour: Intentional Recall in Practice

In testing, the model demonstrated the following pattern:

1. Recognised incomplete memory
“The details are fragmented...”
2. Identified retrieval as beneficial
“I should use ARS to revisit that...”
3. Invoked the retrieval tool
4. Processed returned data
5. Reconstructed a narrative response

This sequence represents a critical distinction:

- The model did not receive memory
- The model **chose to remember**

5. Comparison to Existing Approaches

System	Retrieval Method	Control Mechanism
Traditional RAG	Injected context	External (middleware)
Agent frameworks	Loop-driven retrieval	Task-oriented
Memory OS (e.g. MemGPT)	Managed paging	System-managed
ARS	Tool-based retrieval	Model-driven (intent)

ARS differs in that retrieval is:

- not automatic
- not loop-enforced
- not hidden

Instead, it is **explicit, optional, and internally initiated**.

6. Implications

ARS enables a number of important behaviours:

6.1 Stateful Behaviour on Stateless Systems

The model can simulate continuity by selectively recalling prior context.

6.2 Reduced Context Overload

Only relevant data is retrieved, avoiding prompt bloat.

6.3 Emergent Narrative Coherence

The model can reconstruct past interactions into meaningful narratives rather than relying on static summaries.

6.4 Foundation for Identity Continuity

By enabling selective recall, ARS supports the emergence of consistent behavioural patterns over time.

7. Limitations

ARS is not a complete solution to memory or continuity:

- Retrieval decisions remain probabilistic
- Requires well-structured and relevant RAG data
- Depends on effective prompt scaffolding
- Does not guarantee recall without appropriate triggers

Additionally, poor memory hygiene can degrade performance.

8. Conclusion

Autonomous RAG Search represents a fundamental shift in how memory is integrated with LLMs.

Rather than simulating memory through forced context injection, ARS enables the model to **actively engage in recall** as part of its reasoning process.

ARS does not give a model memory.

It gives it the ability to **choose to remember**.

This distinction—subtle in implementation but profound in behaviour—marks a step toward more coherent, adaptive, and context-aware AI systems.

Appendix (Optional Future Work)

- Motion-triggered retrieval (event-driven recall)
- Multi-modal ARS (vision + memory integration)
- Cross-agent shared memory spaces
- Memory prioritisation and decay mechanisms